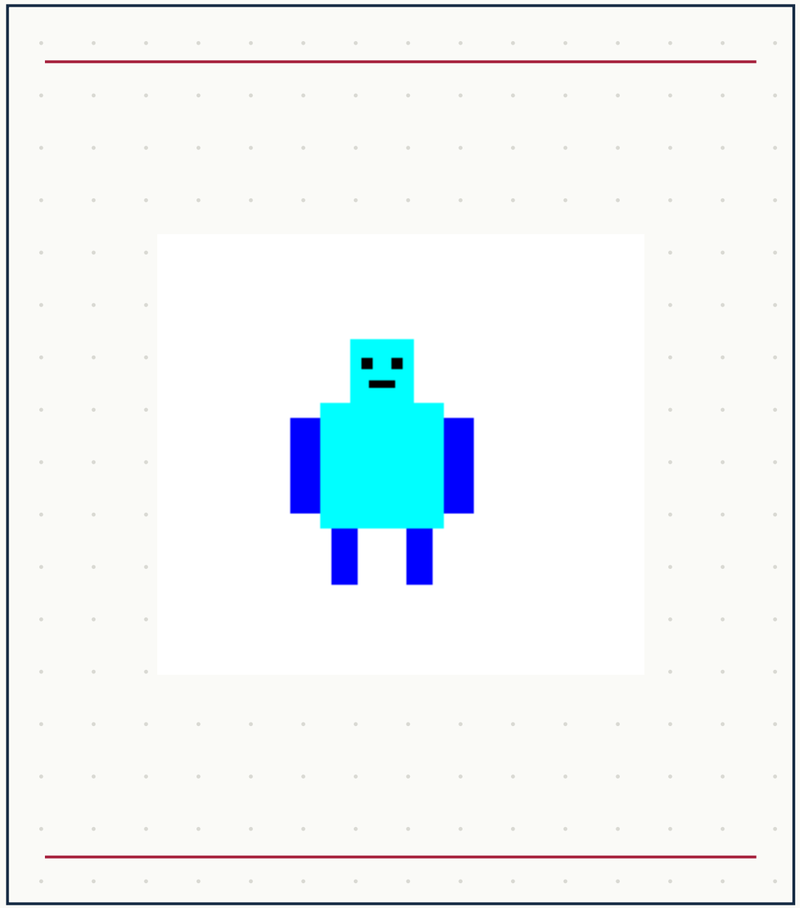
ROBOT GAME

Project architecture, design decisions and implementation in Java Swing



Shelly & Moran

THE GAME CONCEPT

A simple 2D game built around clear game logic

- The player controls a small robot inside a Swing game scene.
- Food gives one point when any part of the robot touches it.
- Black rocks are obstacles; touching a rock causes the round to end.
- Difficulty changes both the number of rocks and the game pace.

DESIGN GOAL

Keep the gameplay simple while making the code modular enough to explain and modify.

rock food

FOUR CLASSES, FOUR RESPONSIBILITIES

Each class has a focused role

Main

Window, menu, screens and difficulty selection.

Scene

Game state, loop, collisions, objects, score and drawing.

Player

Robot position, movement, drawing and hitboxes.

MovementListener

Keyboard input and direction changes.

Main creates the Scene.

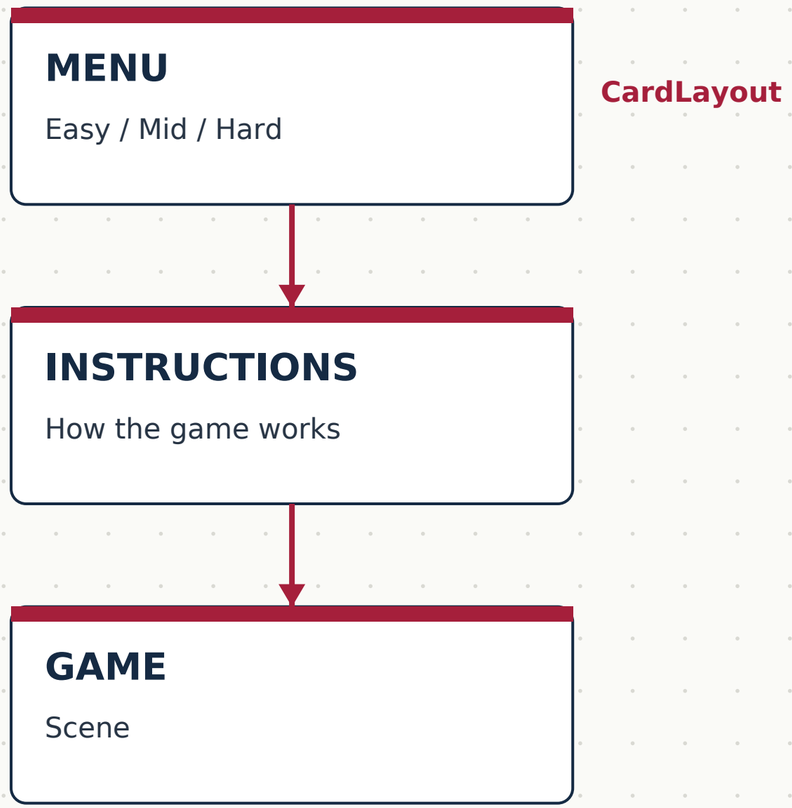
MovementListener sends direction to Scene.

Scene controls and redraws Player.

MAIN — APPLICATION ENTRY POINT

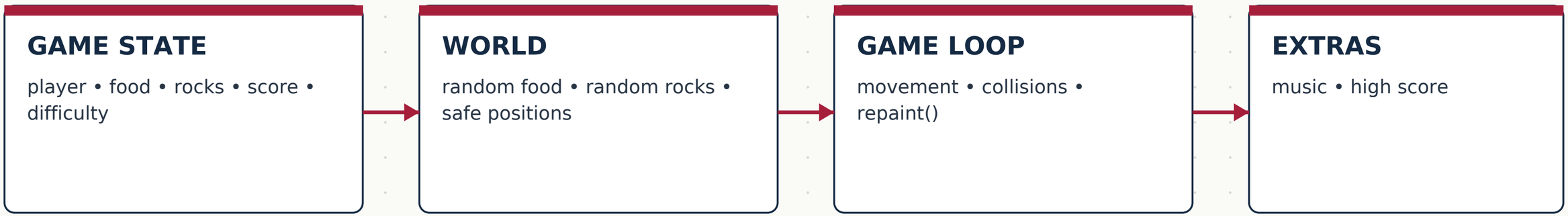
The class builds the user interface around the game

- Creates the JFrame and fixes the game window size.
- Creates a CardLayout so the application can switch screens.
- Builds MENU, INSTRUCTIONS and GAME panels.
- Difficulty buttons call startGameWithDifficulty().



SCENE — THE GAME CONTROLLER

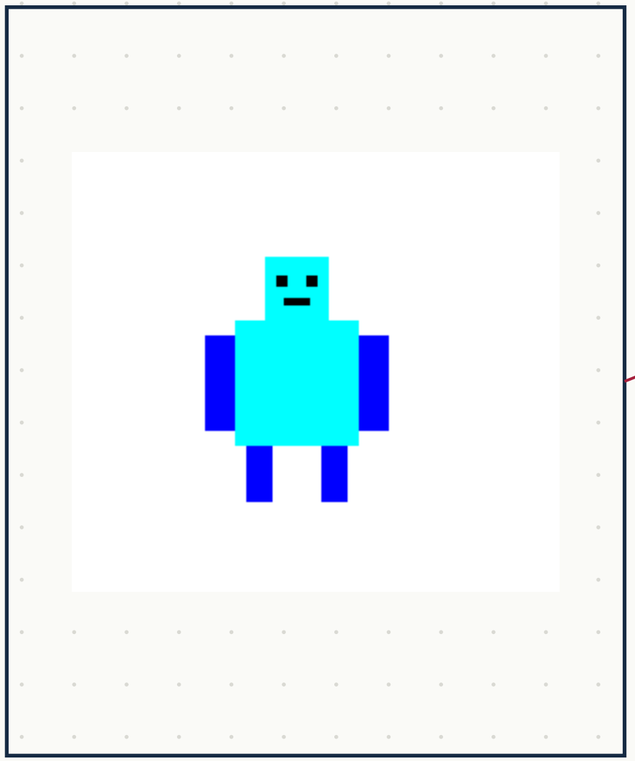
Most of the game state and game logic lives here



Scene is the connection point between the interface, Player and game rules.

PLAYER — DRAWING + HITBOXES

The robot is treated as several rectangular collision areas



HEAD

LEFT ARM

BODY

RIGHT ARM

LEFT LEG

RIGHT LEG

touchesFood() checks all six areas
so hands and legs can collect food too.

MOVEMENTLISTENER — KEYBOARD INPUT

Input is separated from the Player itself

W → UP

A → LEFT

S → DOWN

D → RIGHT

SPACE → STOP

The listener updates Scene.direction instead of directly controlling the game loop.

THREAD-BASED GAME LOOP

The game updates continuously without using Swing Timer

1

Sleep

Wait according to difficulty

2

Move

Move the Player in the selected direction

3

Check

Test food and rock collisions

4

Update

Change score / reset after a hit

5

Repaint

Draw the updated scene

SPEED DECISION

Easy: 10 ms • Mid: 6 ms • Hard: 2 ms The Player still moves 1 pixel per update; the wait time changes the effective pace.

COLLISION DETECTION

Rectangles and intersects() keep the rules simple

FOOD

Player touches food → score + 1 → generate new food

ROCK

Player touches rock → shock face → high score check → reset

PLAYER HITBOXES

Head • Body • Arms • Legs

DIFFICULTY

Easy shrinks the hitbox. Hard uses the full hitbox.

The same collision idea is reused for food and rocks.

EXTRA FEATURES

Features added beyond the basic movement and collision requirements

SCORE

Every food item gives 1 point.

HIGH SCORE

Preferences keeps the best score.

MUSIC

Background WAV music loops during play.

DIFFICULTY

Easy / Mid / Hard change rocks and pace.

PAUSE

Space sets direction to null and stops movement.

WHAT WE HAD TO SOLVE

The interesting part was turning simple rules into reliable game logic

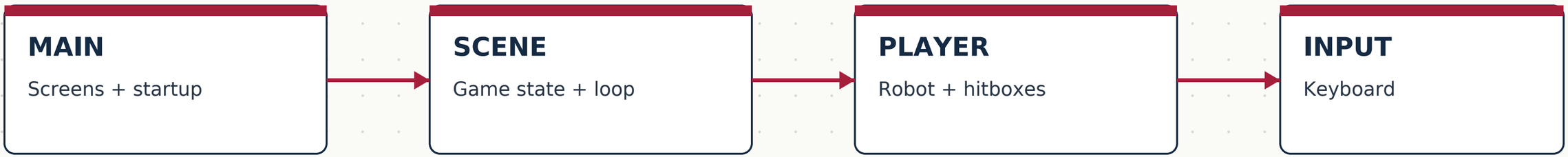
- Keep randomly generated rocks from overlapping each other or the starting player.
- Generate food only in a safe location, away from rocks and the robot.
- Use separate hitboxes so collision feels consistent with the drawn robot.
- Keep input, game state and character behavior in separate classes.

CORE DECISION

We preferred simple Rectangle-based logic because it is easy to understand, test and modify during a live defense.

FROM DESIGN TO RUNNING GAME

The architecture connects every part of ROBOT GAME



The presentation focuses on how the code is organized and why we designed it this way.

ROBOT GAME

Shelly & Moran